

Tutorial XII - Passenger Flow Control in Urban Rail

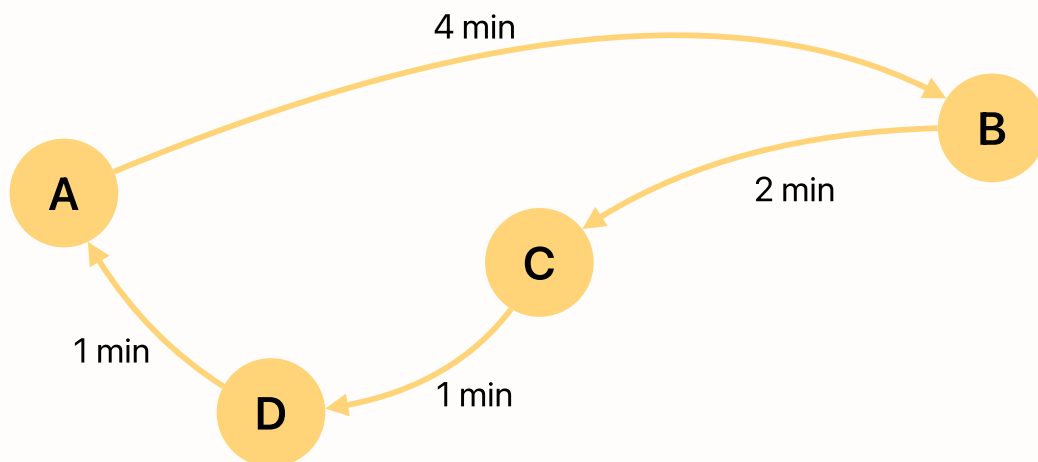
Applied Optimization with Julia

Introduction

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.



Consider the depicted 'Golden Line' on the left with 4 different stations A, B, C, D. For an upcoming timeframe of 10 minutes divided into 10 periods, the transportation demand is going to exceed the available capacity of the network and each origin-destination pair will be requested by at least 1 passenger. To handle the inflow, queues will be put in place at each metro station.

The Challenge

Your task is to minimize the queues without exceeding the available transport capacity of max. 100 passengers per minute of each arc. A safety buffer per arc is not needed. Each station layout is excellent, with sufficient stairs and escalators. Thus, the station

itself is fully capable of handling any inflow that may result from the optimized restricted inflow. Nonetheless, queues are still necessary as the arcs cannot handle each input.

1. Flow Analysis

Suppose we are in minute 7 at the arc of (D,A). Which inflows from which stations are going to impact the flow into the arc in this minute?

Write out the set $\mathcal{R}_{(D,A),7}$ and shortly explain your answer. You can write out the set in a comment, for example like this:

```
#=  
{(A,B,1), (B,C,1), (C,D,1)}  
=#
```

Tip

To solve this task, it might be helpful to work with paper and pen to sketch the problem.

```
#=  
YOUR ANSWER BELOW  
  
=#
```

2. Inflow Control

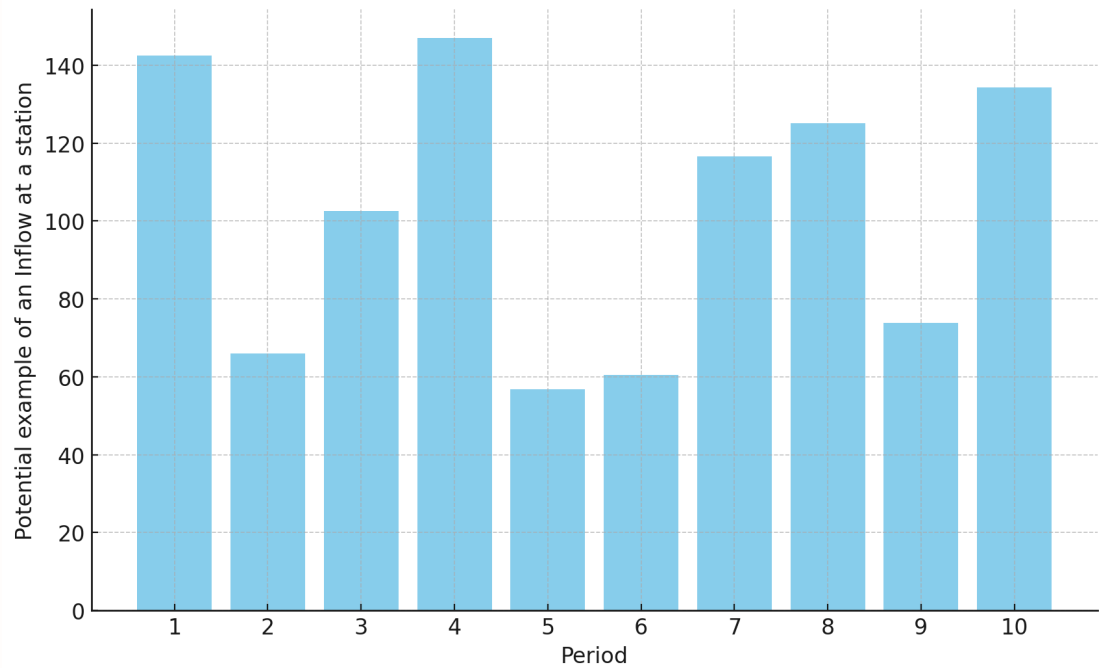


Figure 1: Example: Fluctuations

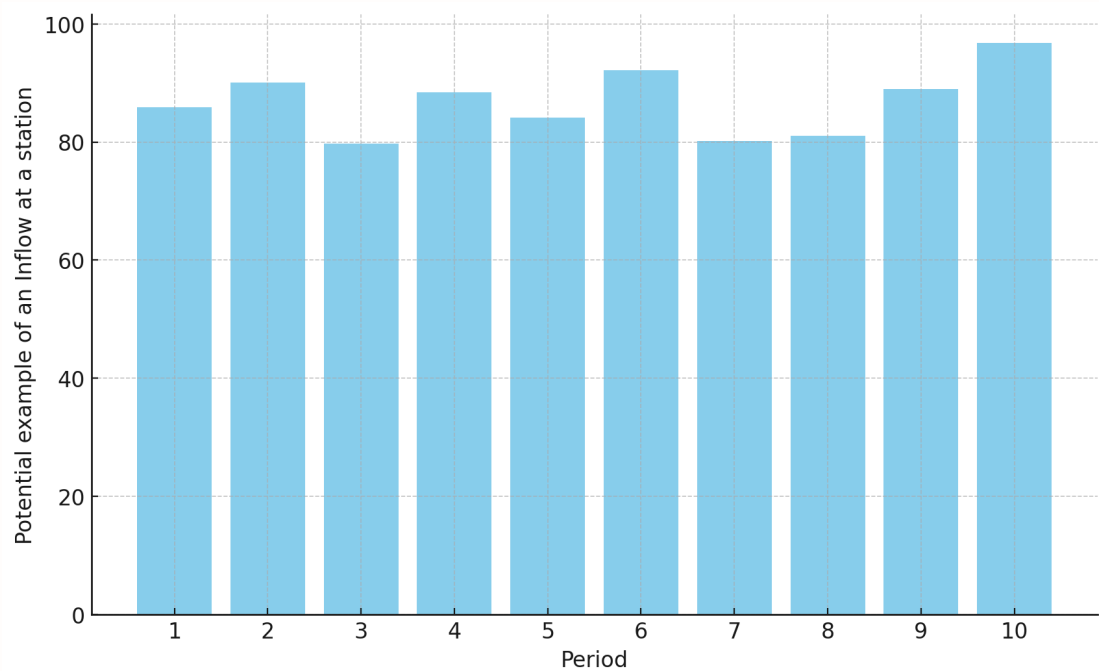


Figure 2: Example: Smoothed

As there is a rather large fluctuation of allowed inflow between periods, you are asked to introduce new constraints for the model. In the first period, the inflow at each station is supposed to be unrestricted. Thereafter, it is maximally allowed to change by 20 persons in both directions per period at each station.

Can you write out the decision variables and the additional constraints for the model as JuMP constraints?

 Tip

You don't need to solve the model or define the objective function. You just need to constraint the fluctuations and add the appropriate variables.

```
using JuMP
metro_model = Model()

# YOUR CODE BELOW
```

3. Bidirectional Flow

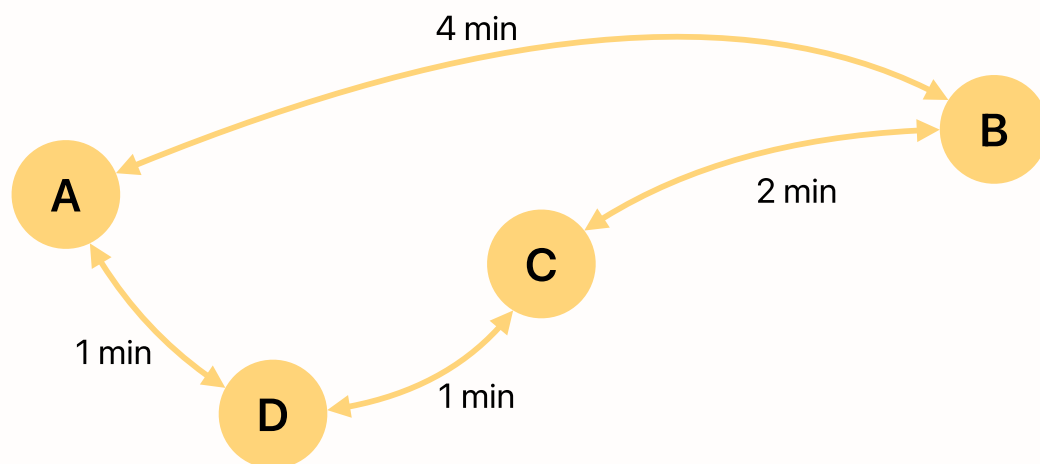


Figure 3: Bidirectional Metro Network

The metro was improved and there is now the possibility to travel in both directions. How would this change the set $\mathcal{R}_{(MD,MA),7}$ from 1.?

Write out the new set $\mathcal{R}_{(D,A),7}$ manually and shortly explain your answer.

```
#=
# YOUR ANSWER BELOW

=#
```

4. Capacity Analysis

Although the system is two-directional now, the overall number of trains of the metro provider has not changed. Would the change from a one-directional metro system to a two-directional metro system decrease the likelihood of crowd-accidents due to insufficient arc-capacities?

Please explain your answer in a few sentences.

```
#=
# YOUR ANSWER BELOW

=#
```

5. Computing the Set $\mathcal{R}_{e,t}$

Can you compute the set $\mathcal{R}_{e,t}$ for the one-directional flow? Generate a dictionary R that contains $e \times t$ entries. Each entry $r_{e,t}$ should contain a vector with all origin-destination pairs and the corresponding time period saved as a tuple. Use the results to check your answer from the first task.

Note

This task can be a bit tricky, as it is a bit of a challenge. But as it is the last tutorial, I figured a small challenge is fine.

```
# YOUR CODE BELOW
```

If you encounter any difficulties and cannot solve the problem, please document your issues here:

```
#=

=#
```

6. Bonus: Metro Flow Simulation (0.5 points)

Now, you can earn up to 0.5 additional bonus points by building a simulation of passenger flows through a metro network. The network consists of two lines with a shared transfer station. Passengers enter at stations A, B, C, D and travel to various destinations.

First, study the network data below to understand the structure and then implement a minute-by-minute simulation that tracks the queues at entry stations, passenger flows through arcs and arc utilization over time. Then, visualize the results with some plots and try to implement inflow regulation to achieve a feasible solution (no arc exceeds capacity).

Tip

Good luck! This is really a tough task.

```

# Stations and arcs
stations = [:A, :B, :C, :D, :T, :E, :F]
entry_stations = [:A, :B, :C, :D]
arcs = [(:A, :B), (:B, :T), (:T, :E), (:C, :D), (:D, :T), (:T, :F)]

# Network parameters
travel_time = 1 # minutes per arc
arc_capacity = 90 # passengers per minute
n_minutes = 20 # total simulation time

# Inflow regulation: single constant rate per station (passengers/minute)
# Set to 1000 (no regulation) initially - experiment to find feasible values
inflow_rates = Dict{:A => 1000, :B => 1000, :C => 1000, :D => 1000}

# Paths for each OD pair
paths = Dict{
  (:A, :B) => [(:A, :B)],
  (:A, :E) => [(:A, :B), (:B, :T), (:T, :E)],
  (:A, :F) => [(:A, :B), (:B, :T), (:T, :F)],
  (:B, :E) => [(:B, :T), (:T, :E)],
  (:B, :F) => [(:B, :T), (:T, :F)],
  (:C, :D) => [(:C, :D)],
  (:C, :F) => [(:C, :D), (:D, :T), (:T, :F)],
  (:C, :E) => [(:C, :D), (:D, :T), (:T, :E)],
  (:D, :F) => [(:D, :T), (:T, :F)],
  (:D, :E) => [(:D, :T), (:T, :E)]
}

# Demand data: demand[period][(origin, destination)] = passengers
demand = Dict{
  1 => Dict{(:A,:B)=>7, (:A,:E)=>12, (:A,:F)=>5, (:B,:E)=>15, (:B,:F)=>10,
    (:C,:D)=>5, (:C,:F)=>10, (:C,:E)=>8, (:D,:F)=>15, (:D,:E)=>12},
  2 => Dict{(:A,:B)=>8, (:A,:E)=>15, (:A,:F)=>10, (:B,:E)=>18, (:B,:F)=>15,
    (:C,:D)=>8, (:C,:F)=>15, (:C,:E)=>10, (:D,:F)=>18, (:D,:E)=>15},
  3 => Dict{(:A,:B)=>12, (:A,:E)=>22, (:A,:F)=>18, (:B,:E)=>28, (:B,:F)=>22,
    (:C,:D)=>12, (:C,:F)=>22, (:C,:E)=>15, (:D,:F)=>28, (:D,:E)=>18},
  4 => Dict{(:A,:B)=>15, (:A,:E)=>28, (:A,:F)=>22, (:B,:E)=>32, (:B,:F)=>28,
    (:C,:D)=>15, (:C,:F)=>28, (:C,:E)=>27, (:D,:F)=>32, (:D,:E)=>22},
  5 => Dict{(:A,:B)=>15, (:A,:E)=>28, (:A,:F)=>22, (:B,:E)=>32, (:B,:F)=>28,
    (:C,:D)=>15, (:C,:F)=>28, (:C,:E)=>18, (:D,:F)=>32, (:D,:E)=>22},
  6 => Dict{(:A,:B)=>12, (:A,:E)=>22, (:A,:F)=>18, (:B,:E)=>22, (:B,:F)=>18,
    (:C,:D)=>12, (:C,:F)=>22, (:C,:E)=>15, (:D,:F)=>22, (:D,:E)=>21},
  7 => Dict{(:A,:B)=>8, (:A,:E)=>15, (:A,:F)=>10, (:B,:E)=>18, (:B,:F)=>15,
    (:C,:D)=>8, (:C,:F)=>15, (:C,:E)=>10, (:D,:F)=>18, (:D,:E)=>15},
  8 => Dict{(:A,:B)=>5, (:A,:E)=>10, (:A,:F)=>5, (:B,:E)=>15, (:B,:F)=>10,
    (:C,:D)=>5, (:C,:F)=>10, (:C,:E)=>5, (:D,:F)=>15, (:D,:E)=>10}
}

```

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography